

A faint, light blue watermark in the background features the Python logo (two interlocking snakes) with a circular arrow around it, indicating a cycle or process.

Jakość i Testowanie Kodu w Pythonie

Filip Zając, Eryk Knap

| Plan Prezentacji



Fundamenty

Dlaczego jakość ma znaczenie i koszt błędu w cyklu życia projektu.



Tooling 2025

Przegląd nowoczesnych narzędzi:
Ruff, uv, Pytest i Mypy.



Zaawansowane

Testy mutacyjne, property-based testing i automatyzacja CI/CD.

| Dlaczego Jakość to Podstawa?

Jakość kodu w Pythonie to nie tylko estetyka. To **bezpieczeństwo finansowe** i operacyjne Twojego projektu.

- ✓ **Utrzymywalność:** Łatwiejsze wprowadzanie zmian przez nowe osoby w zespole.
- ✓ **Skalowalność:** Systemy oparte na solidnych testach nie "pękają" pod obciążeniem zmian.
- ✓ **Zaufanie:** Pewność, że refaktoryzacja nie zepsuje kluczowych funkcji biznesowych.



| Reguła 1-10-100: Koszt Błędu

1x

Development

Błąd wykryty przez linter/testy
jednostkowe.

10x

QA / Staging

Wymaga zgłoszenia, odtworzenia i
poprawki.

100x

Produkcja

Ryzyko utraty danych, wizerunku i
klientów.

Im wcześniej wykryjesz anomalię, tym tańsza jest jej eliminacja.

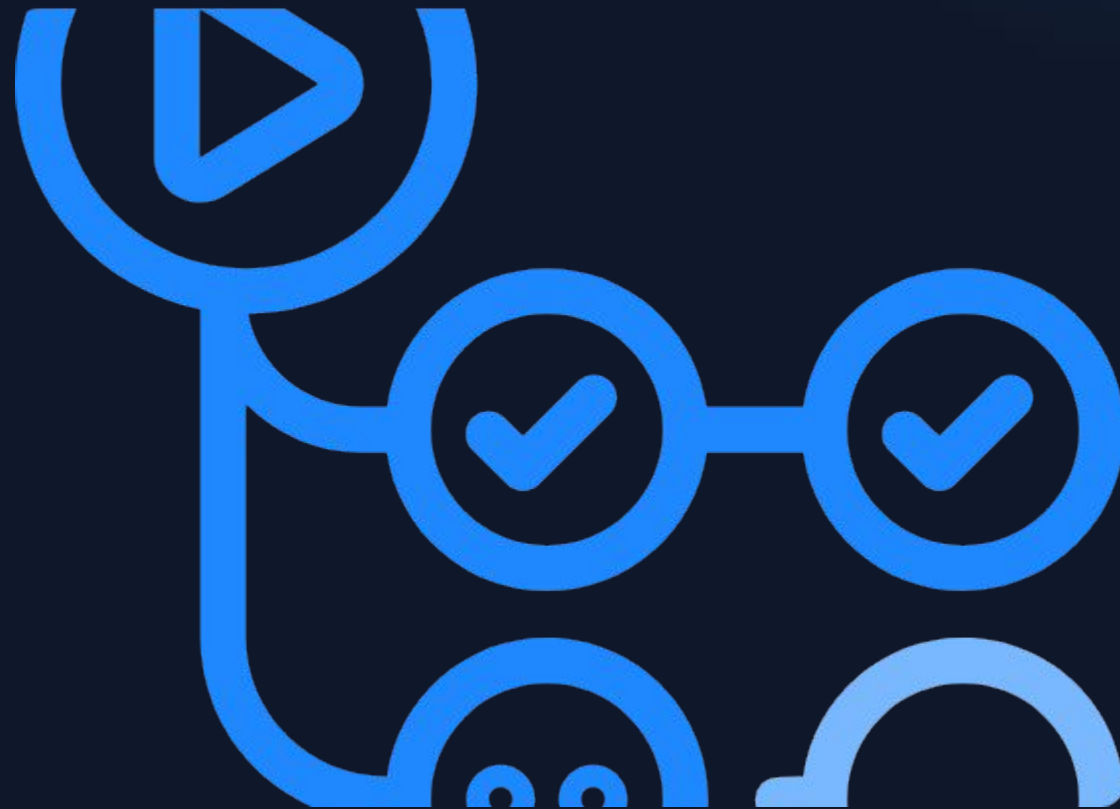
| PEP 8 i Czytelność Kodu

Standard to wspólny język

Python posiada oficjalny przewodnik stylu – **PEP 8**. Jego przestrzeganie sprawia, że kod napisany przez różne osoby wygląda spójnie.

```
# Dobry styl
def calculate_area(radius: float) -> float:
    return 3.14 * radius ** 2
```

Czytelność jest kluczowa: "Code is read much more often than it is written."



| Rewolucja "Ruff & uv"

Rok 2025 należy do narzędzi napisanych w **Rust**. Zastępują one wolne, legacy'owe rozwiązania.

⚡ **Ruff:** Linter i formatter, który jest 10-100x szybszy niż Flake8 czy Black.

📦 **uv:** Unified package manager. Zastępuje pip, poetry i pyenv w jednym binarnym pliku.

🛡️ **Bezpieczeństwo:** Deterministic lockfiles jako standard.



RUST

| Ruff: Jeden by wszystkimi rządzić

Zastępuje 50+ pluginów

Ruff to nie tylko linter. To kompleksowe narzędzie, które zastępuje m.in.: **isort**, **flake8**, **bandit**, **pyupgrade**, **pydocstyle**.

Konfiguracja w pyproject.toml

Całość konfiguracji mieści się w jednym pliku, co ułatwia synchronizację ustawień w dużych zespołach deweloperskich.

Zalety: Natychmiastowy feedback w IDE (VS Code / PyCharm).

| uv: Przyszłość Package Managementu

Koniec z wieloma narzędziami

Uv to rewolucja stworzona przez Astral. Pozwala na:

- ✓ Błyskawiczną instalację paczek (milisekundy).
- ✓ Zarządzanie wersjami Pythona (brak potrzeby pyenv).
- ✓ Tworzenie odizolowanych venv'ów w locie.



| Typowanie: Mypy i Pylance

-  **Wykrywanie błędów logicznych:** Narzędzia do statycznej analizy typów wyłapują błędy *zanim* kod zostanie uruchomiony.
-  **Mypy:** Klasyka i standard rynkowy. Bardzo rygorystyczny, wspiera zaawansowane konstrukcje generyczne.
-  **Pylance:** Stworzony przez Microsoft, zintegrowany z VS Code (Pylance). Jest znacznie szybszy w dużych bazach kodu.
-  **Typy jako dokumentacja:** Adnotacje typów informują innych programistów, jakich danych wejściowych oczekuje funkcja.

| Pydantic i Walidacja Danych

Parse, don't validate

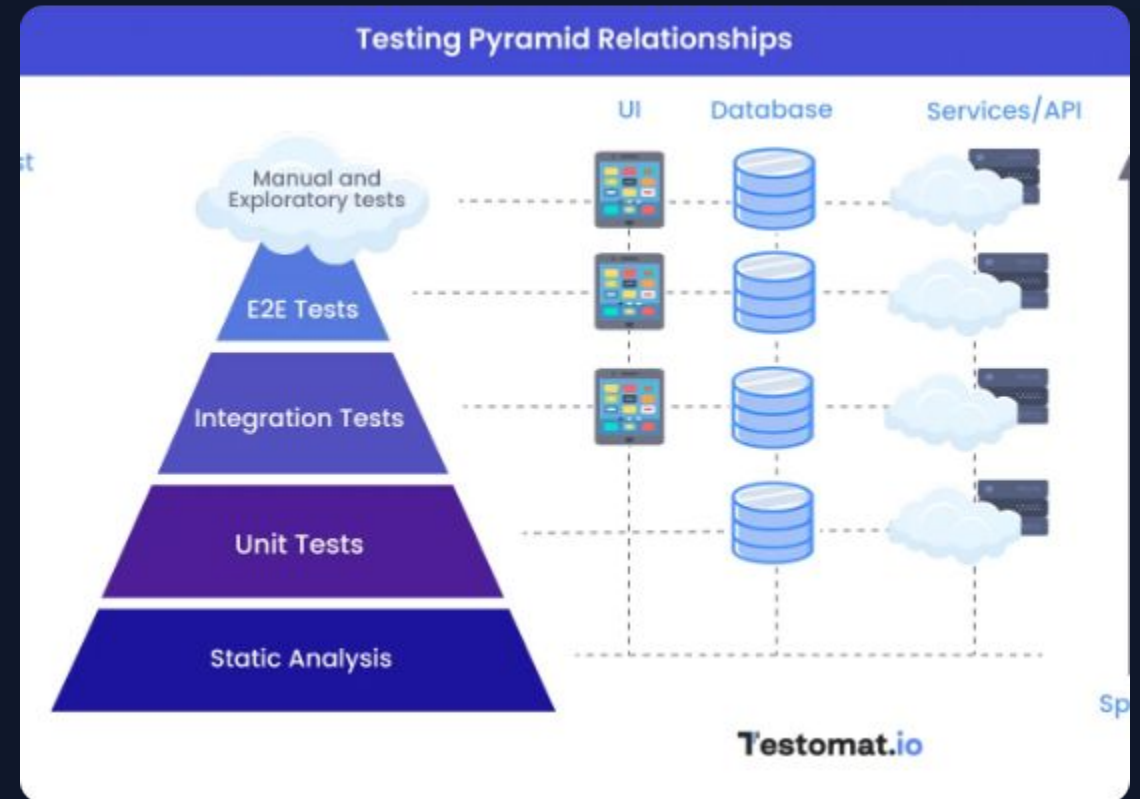
Pydantic gwarantuje, że dane wewnątrz Twojej aplikacji są zgodne ze schematem. Jeśli dane są niepoprawne, aplikacja wyrzuci błąd na wejściu.

Integracja z FastAPI

Standard w budowaniu API. Automatycznie generuje dokumentację OpenAPI na podstawie Twoich modeli danych.

| Piramida Testów 2.0

-  **Unit Tests (70%):** Szybkie, tanie, testują logikę w izolacji.
-  **Integration Tests (20%):** Sprawdzają połączenia z bazą danych i zewnętrznymi API.
-  **E2E / UI Tests (10%):** Najdroższe, symulują pełną ścieżkę użytkownika.



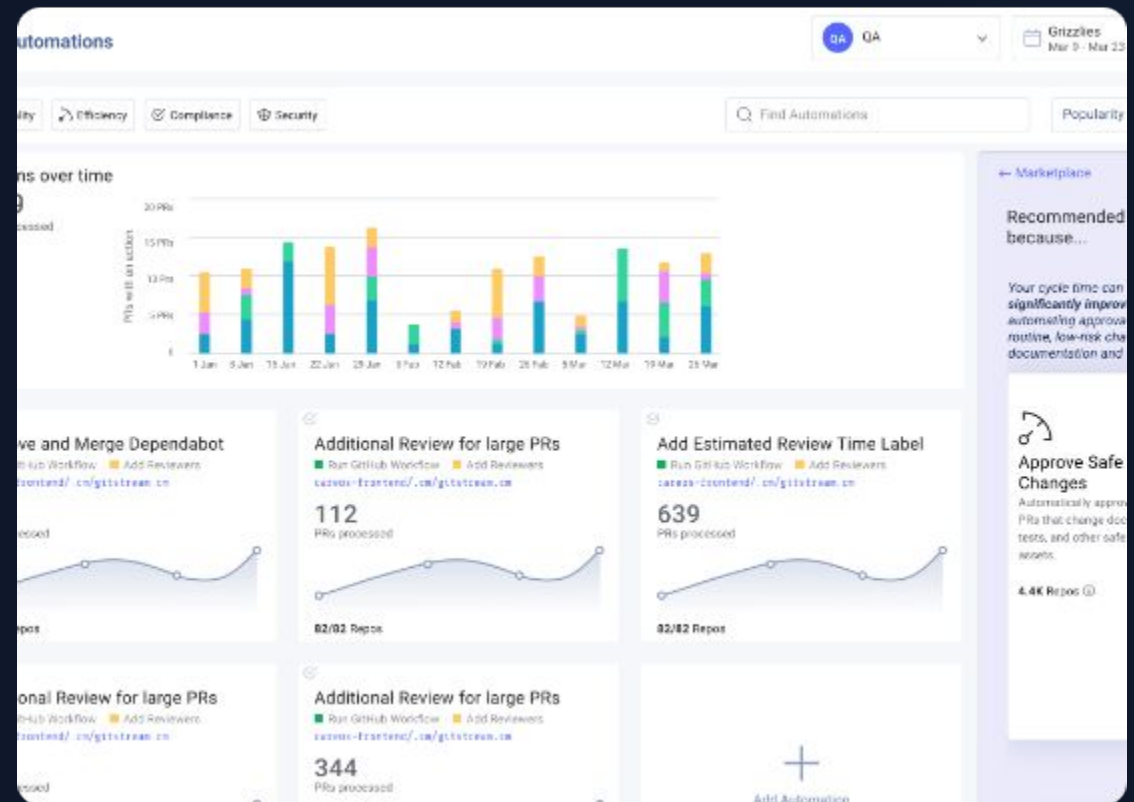
| Standard De Facto: Pytest

Mniej kodu, więcej mocy

W przeciwieństwie do wbudowanego unittest, Pytest pozwala na używanie zwykłych instrukcji assert.

```
def test_add():  
    assert add(2, 3) == 5
```

Ekosystem pluginów (pytest-cov, pytest-xdist) czyni go bezkonkurencyjnym.



| Magia Zarządzania Stanem

-  **Fixtures:** Modularne funkcje dostarczające dane lub zasoby do testów (np. połączenie z bazą).
-  **Reużywalność:** Jedna definicja fixture'a może być użyta w setkach testów.
-  **Scope:** Możesz definiować zasięg (session, module, function), co optymalizuje czas testów.
-  **Teardown:** Automatyczne sprzątanie po teście (np. zamykanie bazy danych).

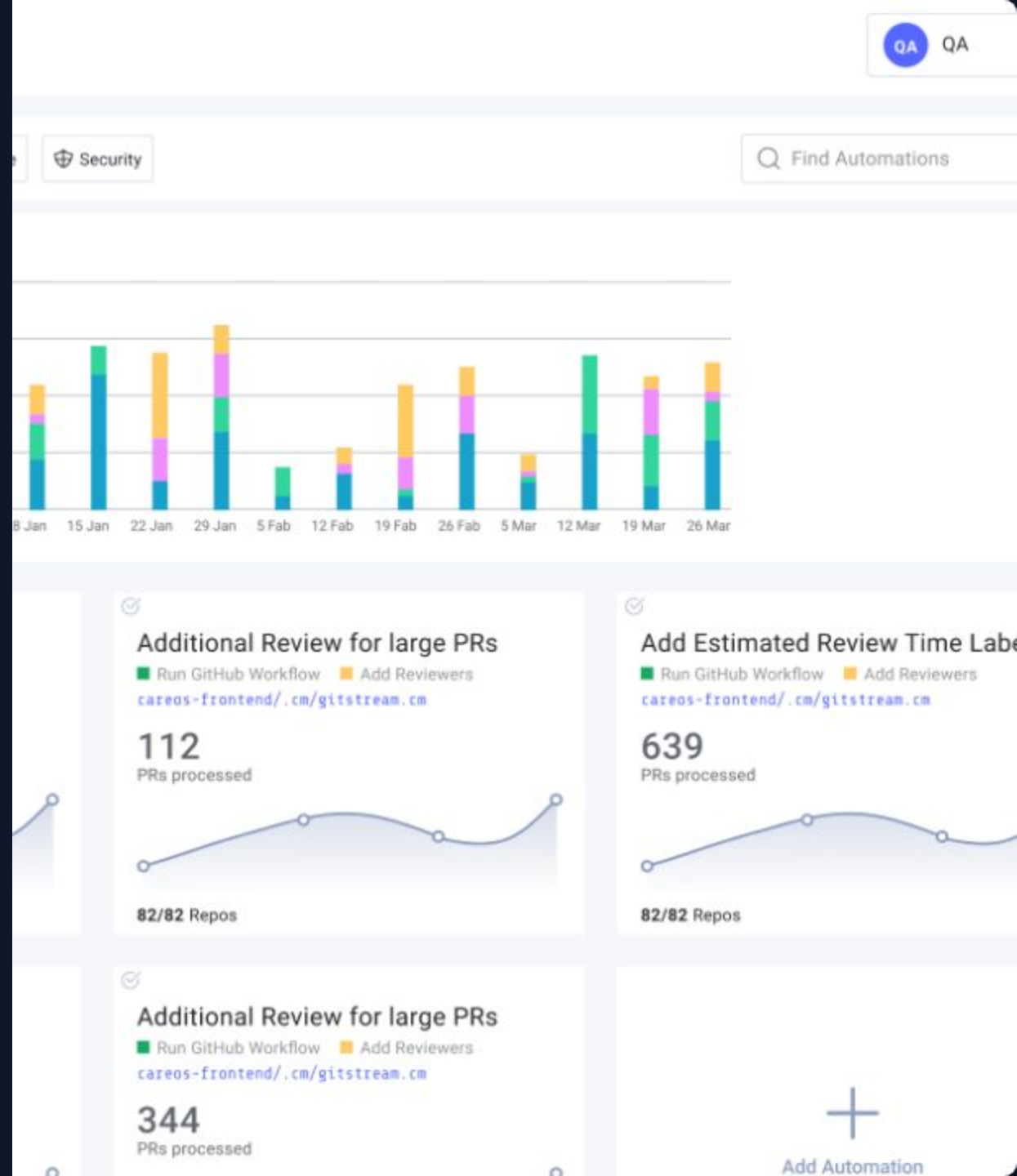
DRY w Testach

Parametryzacja pozwala na uruchomienie jednego testu z wieloma zestawami danych.

Zalety

:

- Eliminacja duplikacji kodu testowego.
- Lepsza przejrzystość przypadków brzegowych.
- Każdy zestaw danych raportowany jest jako oddzielny test.



| Izolacja i Determinizm



Mockowanie: Zastępowanie rzeczywistych komponentów (np. API Stripe) obiektami udającymi ich zachowanie.



Bezpieczeństwo: Twoje testy nie powinny wysyłać prawdziwych e-maili ani obciążać karty kredytowej.



Prędkość: Testy z mockami wykonują się w milisekundach, bo nie oczekują na odpowiedź sieci.



Kiedy unikać? Nadmierne mockowanie może ukryć błędy integracyjne.

| Bazy Danych i API

Aspekt	Testy Jednostkowe (Mock)	Testy Integracyjne (Baza)
Czas wykonania	Ekstremalnie szybkie	Szybkie / Średnie
Wiarygodność	Weryfikują tylko logikę	Weryfikują realne zapytania SQL
Zależności	Brak	Wymagany Docker / SQLite

| Playwright: Symulacja Użytkownika

Testowanie całego systemu

Playwright to nowoczesne narzędzie do testowania UI. Pozwala na automatyczne klikanie w przeglądarce i sprawdzanie efektów.

- Wsparcie dla Chromium, Firefox i WebKit.
- Auto-waiting: Brak potrzeby ustawiania sleep().
- Nagrywanie wideo i robienie screenshotów błędów.



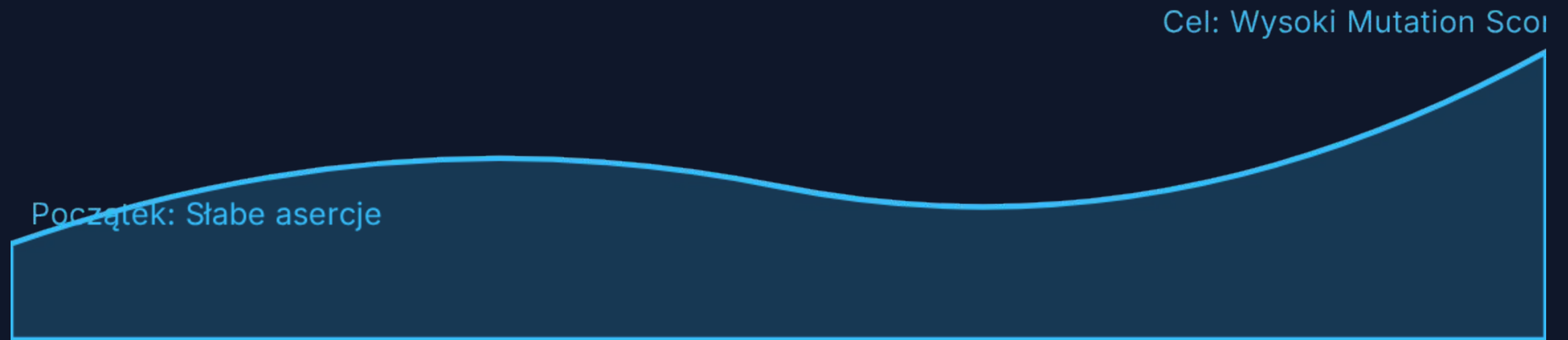
| Czy 100% Coverage ma sens?



"Coverage to metryka przydatna do znalezienia nieprzetestowanego kodu, a nie dowód na jego poprawność."

| Sprawdzanie Jakości Testów

Testowanie mutacyjne (np. **Mutmut**) celowo wprowadza błędy do Twojego kodu (mutanty). Jeśli testy nadal przechodzą pomyślnie, oznacza to, że Twoje asercje są słabe.



Wskaźnik mutacyjny = (Zabite mutanty / Wszystkie mutanty) * 100%

| Property-Based Testing

Zamiast wymyślać dane...

Narzędzie **Hypothesis** samo generuje tysiące przypadków testowych, próbując znaleźć ten jeden, który zepsuje Twoją funkcję.

Wykrywanie Edge Cases

Idealne do testowania algorytmów, parsowania danych i operacji matematycznych, gdzie ludzka wyobraźnia zawodzi.

| MkDocs i Dokumentacja jako Kod



Dokumentacja techniczna: Powinna żyć w repozytorium (Markdown).



MkDocs-Material: Standard wizualny dokumentacji w Pythonie. Generuje piękne, responsywne strony.



Automatyczna aktualizacja: Pipeline CI buduje dokumentację przy każdym merge'u do maina.

| Bramkarz Jakości: Pre-commit Hooks



Stop przed błędem

Pre-commit uniemożliwia zapisanie kodu, jeśli Ruff lub Mypy zgłoszą błąd.



Auto-fix

Automatycznie usuwa niepotrzebne importy i formatuje kod przy każdym komicie.



Czysta historia

Dzięki pre-commit w historii Gita znajdują się tylko "ładne" i poprawne zmiany.

| GitHub Actions w Praktyce



Push

Deweloper wysła kod.



Lint

Ruff sprawdza styl.



Test

Pytest uruchamia testy.



Audit

Bandit sprawdza bezpieczeństwo.



Deploy

Wdrożenie po sukcesie.

| Strategia Refaktoryzacji



Reguła Skauta: Zostaw kod nieco lepszym, niż go zastałeś.



Usuwanie długu: Planuj dedykowany czas (np. 1 dzień w sprincie) na poprawę jakości.



SonarQube: Monitoruj "smell" w kodzie i dług techniczny w czasie rzeczywistym.

A dimly lit office meeting room. A man in a light-colored shirt stands at the front, presenting to a group of people seated around a large, curved wooden table. Several laptops are open on the table, and some people are looking at them. The room has a modern feel with a large screen in the background and shelves on the wall. The text "Dziękuję za uwagę!" is overlaid in the center.

Dziękuję za uwagę!

| Źródła:



https://static.vecteezy.com/system/resources/previews/071/063/178/non_2x/person-coding-on-laptop-computer-dark-workspace-free-photo.jpg

Source: www.vecteezy.com



<https://icon.icepanel.io/Technology/svg/GitHub-Actions.svg>

Source: techicons.dev



https://miro.medium.com/v2/resize:fit:667/1*1E7HToSYCfIK0rjBTNy6Lg.png

Source: medium.com



<https://provenit.com/wp-content/uploads/2026/04/meeting.webp>

Source: provenit.com



Thumbnail
for

<http://testomat.io/wp-content/webp-express/webp-images/uploads/2025/04/Structure-Testing-Pyramid-Relationships.png.webp>

Source: testomat.io



https://assets.linearb.io/image/upload/v1737049459/SEI_Automation_Dashboard_280c19c0d7.png

Source: linearb.io

| Źródła:



Thumbnail
for
[pngtree.com](https://png.pngtree.com/)

https://png.pngtree.com/png-vector/20260217/ourlarge/pngtree-shield-icon-with-checkmark-symbolizing-security-protection-and-verification-in-futuristic-png-image_18790142.webp

Source: [pngtree.com](https://png.pngtree.com/)



https://decodo.com/cdn-cgi/image/width=1280,quality=70,format=auto/https://images.decodo.com/Python_requests_retry_main_illustration_105f0ad66e/Python_requests_retry_main_illustration_105f0ad66e.webp

Source: decodo.com